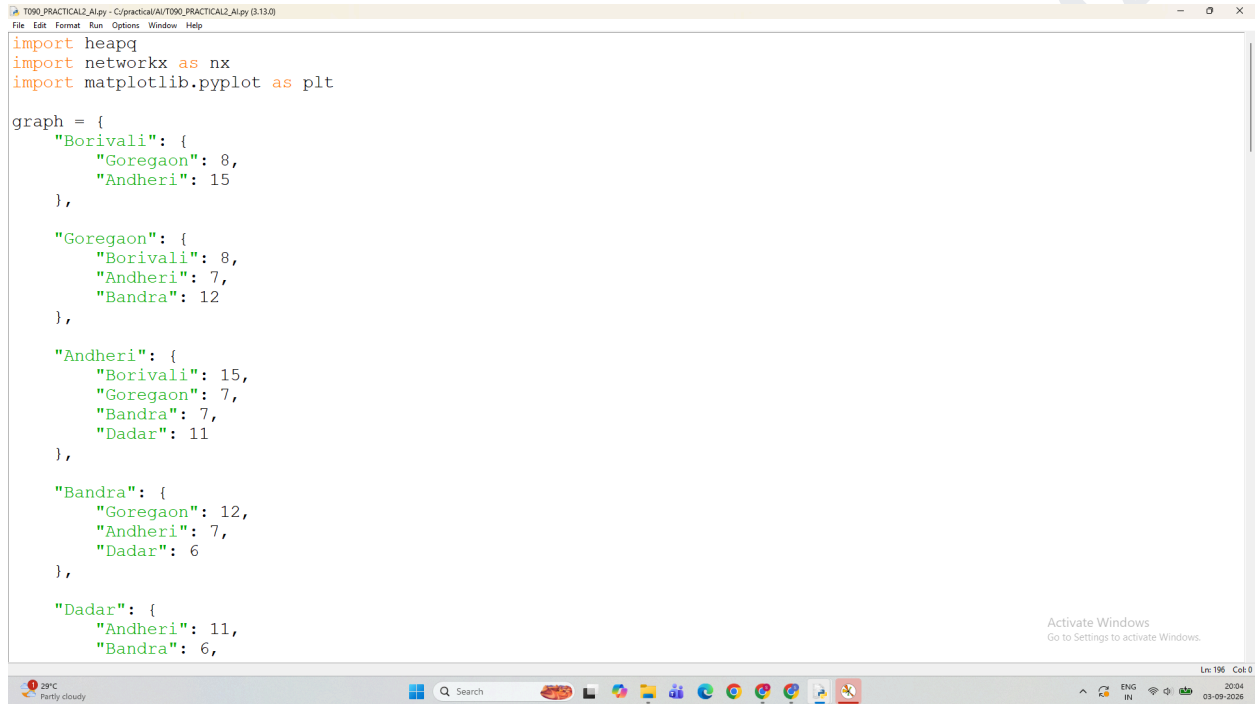


Aim :- A* Search and Recursive Best-First Search

1. Implement the A* Search algorithm for solving a pathfinding problem.
2. Implement the Recursive Best-First Search algorithm for the same problem.
3. Compare the performance and effectiveness of both algorithms.

CODE:A*algorithm



```
import heapq
import networkx as nx
import matplotlib.pyplot as plt

graph = {
    "Borivali": {
        "Goregaon": 8,
        "Andheri": 15
    },
    "Goregaon": {
        "Borivali": 8,
        "Andheri": 7,
        "Bandra": 12
    },
    "Andheri": {
        "Borivali": 15,
        "Goregaon": 7,
        "Bandra": 7,
        "Dadar": 11
    },
    "Bandra": {
        "Goregaon": 12,
        "Andheri": 7,
        "Dadar": 6
    },
    "Dadar": {
        "Andheri": 11,
        "Bandra": 6
    }
}
```

The screenshot shows a code editor window titled 'T090_PRACTICAL2_Alpy - C:\practical\AI\T090_PRACTICAL2_Alpy (3.13.0)'. The code defines a graph with five nodes: Borivali, Goregaon, Andheri, Bandra, and Dadar. Each node is represented as a dictionary of its neighbors and the edge weights. The graph is undirected. The code imports the heapq, networkx, and matplotlib.pyplot modules. The graph is defined as a dictionary where each key is a node name and the value is a dictionary of its neighbors and the edge weights. The nodes and their connections are: Borivali (Goregaon: 8, Andheri: 15), Goregaon (Borivali: 8, Andheri: 7, Bandra: 12), Andheri (Borivali: 15, Goregaon: 7, Bandra: 7, Dadar: 11), Bandra (Goregaon: 12, Andheri: 7, Dadar: 6), and Dadar (Andheri: 11, Bandra: 6). The code is written in Python 3.13.0. The Windows taskbar is visible at the bottom, showing the date as 03-09-2026 and the time as 20:04.

SHETH L.U.J. AND SIR M.V. COLLEGE

```
T090_PRACTICAL2_Alpy - C:\practical\AI\T090_PRACTICAL2_Alpy (3.13.0)
File Edit Format Run Options Window Help

"Mumbai Central": {
    "Dadar": 7,
    "Churchgate": 4
},
"Churchgate": {
    "Dadar": 11,
    "Mumbai Central": 4
}
}

h = {
    "Borivali": 37,
    "Goregaon": 29,
    "Andheri": 24,
    "Bandra": 17,
    "Dadar": 11,
    "Mumbai Central": 4,
    "Churchgate": 0
}

pos = {
    "Borivali": (1, 5),
    "Goregaon": (3, 4),
    "Andheri": (5, 5),
    "Bandra": (5, 3),
    "Dadar": (7, 3),
    "Mumbai Central": (8, 1),
    "Churchgate": (10, 2)
}

def astar(start, goal):
    pq = [(h[start], start, [start], 0)]
    visited = set()
    while pq:
        f, node, path, g = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            return path, g
        for nxt, cost in graph[node].items():
            if nxt not in visited:
                new_g = g + cost
                new_f = new_g + h[nxt]
                heapq.heappush(
                    pq,
                    (new_f, nxt, path + [nxt], new_g)
                )
```

Activate Windows
Go to Settings to activate Windows.

Ln: 196 Col: 9

23°C
Partly cloudy

Search

ENG
IN

20:04
03-09-2026

```
T090_PRACTICAL2_Alpy - C:\practical\AI\T090_PRACTICAL2_Alpy (3.13.0)
File Edit Format Run Options Window Help

def astar(start, goal):
    pq = [(h[start], start, [start], 0)]
    visited = set()
    while pq:
        f, node, path, g = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            return path, g
        for nxt, cost in graph[node].items():
            if nxt not in visited:
                new_g = g + cost
                new_f = new_g + h[nxt]
                heapq.heappush(
                    pq,
                    (new_f, nxt, path + [nxt], new_g)
                )
```

Activate Windows
Go to Settings to activate Windows.

Ln: 196 Col: 9

23°C
Partly cloudy

Search

ENG
IN

20:04
03-09-2026

```
T090_PRACTICAL2_Alpy - C:\practical\AI\T090_PRACTICAL2_Alpy (3.13.0)
File Edit Format Run Options Window Help

    return None, float("inf")

path_a, distance_a = astar(
    "Borivali",
    "Churchgate"
)

print("A* Search")
print("Optimal Path Found:")
print(" -> ".join(path_a))
print("Total Distance =", distance_a, "km")

G = nx.Graph()
for u in graph:
    for v, w in graph[u].items():
        if not G.has_edge(u, v):
            G.add_edge(
                u,
                v,
                weight=w
            )

plt.figure(figsize=(12, 8))

nx.draw_networkx_nodes(
    G,
    pos,
    node_color="lightblue",
    node_size=2500
)

nx.draw_networkx_edges(
    G,
    pos,
    edge_color="gray"
)

path_edges = list(
    zip(path_a, path_a[1:])
)

nx.draw_networkx_edges(
    G,
    pos,
    edgelist=path_edges,
    edge_color="green",
    width=4
)

Activate Windows
Go to Settings to activate Windows.

Ln: 196 Col: 9
23°C
Partly cloudy
Search
ENG
IN
20:04
03-09-2026
```

```
T090_PRACTICAL2_Alpy - C:\practical\AI\T090_PRACTICAL2_Alpy (3.13.0)
File Edit Format Run Options Window Help

plt.figure(figsize=(12, 8))

nx.draw_networkx_nodes(
    G,
    pos,
    node_color="lightblue",
    node_size=2500
)

nx.draw_networkx_edges(
    G,
    pos,
    edge_color="gray"
)

path_edges = list(
    zip(path_a, path_a[1:])
)

nx.draw_networkx_edges(
    G,
    pos,
    edgelist=path_edges,
    edge_color="green",
    width=4
)

Activate Windows
Go to Settings to activate Windows.

Ln: 196 Col: 9
23°C
Partly cloudy
Search
ENG
IN
20:05
03-09-2026
```

SHETH L.U.J. AND SIR M.V. COLLEGE

```
T090_PRACTICAL2_AI.py - C:/practical/AI/T090_PRACTICAL2_AI.py (3.13.0)
File Edit Format Run Options Window Help

G,
pos,
labels={
    n: f"{n}\n(h={h[n]})"
    for n in G.nodes()
},
font_size=9,
font_weight="bold"
)

nx.draw_networkx_edge_labels(
    G,
    pos,
    edge_labels={
        (u, v): f"{w} km"
        for u, v, w in G.edges(
            data="weight"
        )
    },
    font_color="red"
)

plt.title(
    "A* Search Route - Borivali to Churchgate"
)

plt.axis("off")

plt.show()
```

Activate Windows
Go to Settings to activate Windows.

Ln: 196 Col: 0

29°C
Partly cloudy

Search

ENG
IN

20:05
03-09-2026

OUTPUT:

```
"IDLE Shell 3.13.0"
File Edit Shell Debug Options Window Help

Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/practical/AI/T090_PRACTICAL2_AI.py =====
A* Search
Optimal Path Found:
Borivali -> Goregaon -> Bandra -> Dadar -> Churchgate
Total Distance = 37 km
```

Activate Windows
Go to Settings to activate Windows.

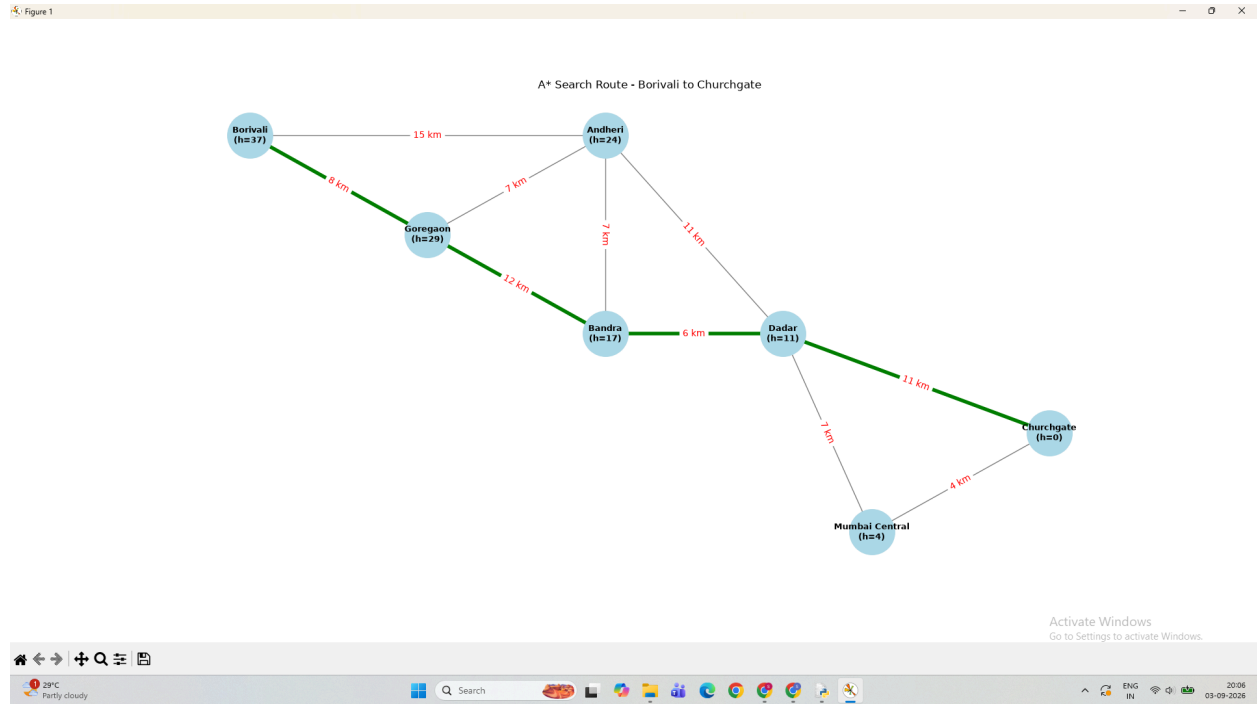
Ln: 9 Col: 0

29°C
Partly cloudy

Search

ENG
IN

20:05
03-09-2026



CODE: Recursive Best-First Search algorithm

```

T090_PRACTICAL2_A1.py - C:\practical\AI\T090_PRACTICAL2_A1.py (3.13.0)
File Edit Format Run Options Window Help
import heapq
import networkx as nx
import matplotlib.pyplot as plt

graph = {
    "Thane": {
        "Mulund": 5,
        "Ghatkopar": 12
    },
    "Mulund": {
        "Thane": 5,
        "Ghatkopar": 7
    },
    "Ghatkopar": {
        "Thane": 12,
        "Mulund": 7,
        "Kurla": 6
    },
    "Kurla": {
        "Ghatkopar": 6,
        "Sion": 5,
        "Dadar": 8
    },
    "Sion": {
        "Kurla": 5,
        "Dadar": 6,
        "CSMT": 12
    },
    "Dadar": {
        "Kurla": 8,
        "Sion": 6,
        "CSMT": 10
    }
}

```

Activate Windows
Go to Settings to activate Windows.

```
T090_PRACTICAL2_Alpy - C:\practical\AI\T090_PRACTICAL2_Alpy (3.13.0)
File Edit Format Run Options Window Help
"CSMT": {
    "Sion": 12,
    "Dadar": 10
}
}
h = {
    "Thane": 33,
    "Mulund": 28,
    "Ghatkopar": 21,
    "Kurla": 15,
    "Sion": 10,
    "Dadar": 10,
    "CSMT": 0
}
pos = {
    "Thane": (1, 5),
    "Mulund": (3, 5),
    "Ghatkopar": (5, 4),
    "Kurla": (6, 3),
    "Sion": (7, 5),
    "Dadar": (8, 3),
    "CSMT": (10, 4)
}

def astar(start, goal):
    pq = [(h[start], start, [start], 0)]
    visited = set()

    while pq:
        f, node, path, g = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            return path, g
        for nxt, cost in graph[node].items():
            if nxt not in visited:
                new_g = g + cost
                new_f = new_g + h[nxt]
                heapq.heappush(
                    pq,
                    (new_f, nxt, path + [nxt], new_g)
                )
        return None, float("inf")

path_a, distance_a = astar(
    "Thane",
```

```

T090_PRACTICAL2_Alpy - C:\practical\AI\T090_PRACTICAL2_Alpy (3.13.0)
File Edit Format Run Options Window Help
print("A* Search")

print("Optimal Path Found:")

print(" -> ".join(path_a))

print(
    "Total Distance =",
    distance_a,
    "km"
)

G = nx.Graph()

for u in graph:
    for v, w in graph[u].items():
        if not G.has_edge(u, v):
            G.add_edge(
                u,
                v,
                weight=w
            )

plt.figure(figsize=(12, 8))

nx.draw_networkx_nodes(
    G,
    pos,
    edge_color="gray"
)

path_edges = list(
    zip(path_a, path_a[1:])
)

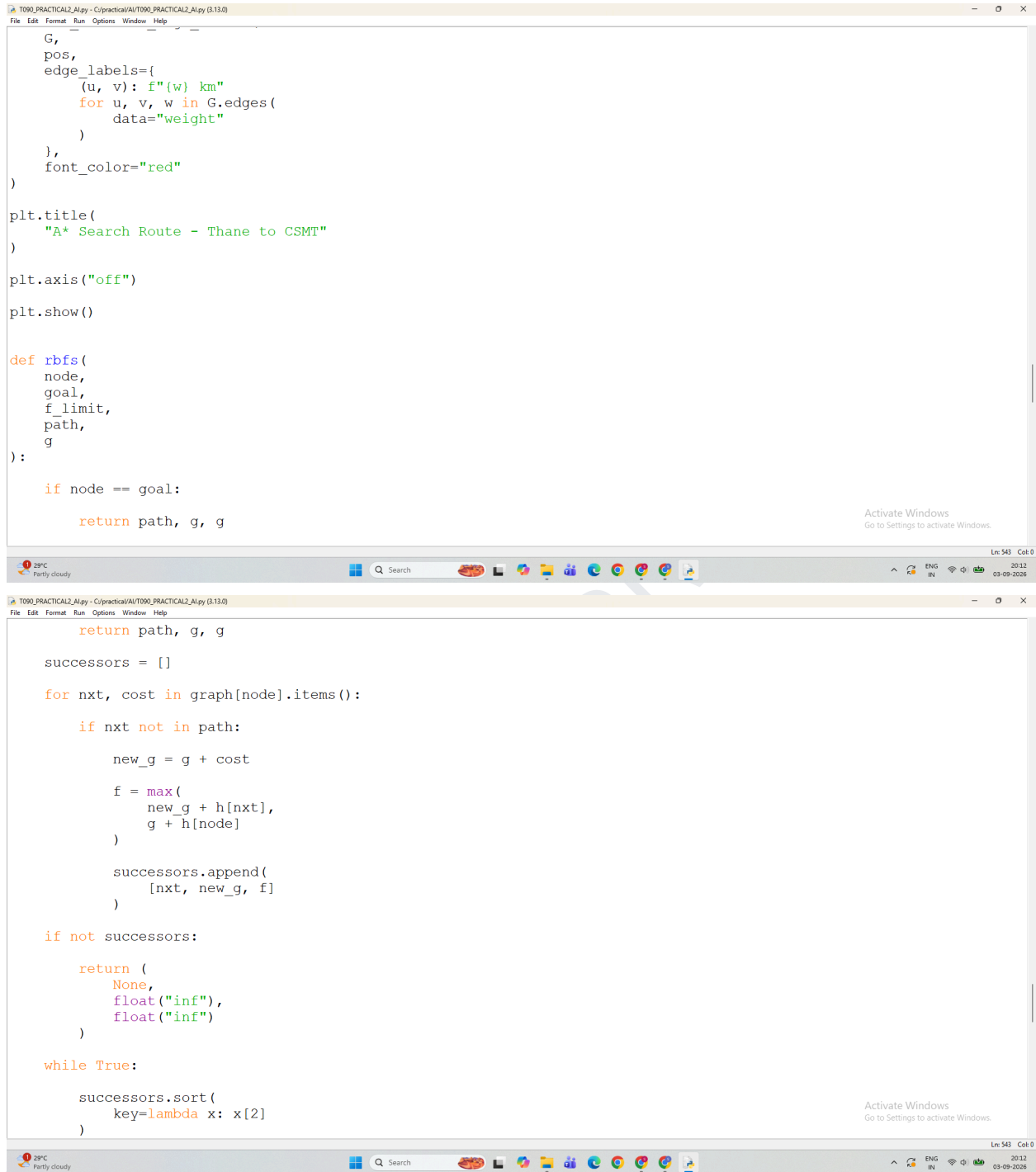
nx.draw_networkx_edges(
    G,
    pos,
    edgelist=path_edges,
    edge_color="green",
    width=4
)

nx.draw_networkx_labels(
    G,
    pos,
    labels={
        n: f"{n}\n(h={h[n]})"
        for n in G.nodes()
    },
    font_size=9,
    font_weight="bold"
)

nx.draw_networkx_edge_labels(
    G,
    pos,
    edge_labels={
        (u, v): w
        for u, v, w in graph.items()
    },
    font_size=8,
    font_weight="normal"
)

plt.show()

```



```

G,
pos,
edge_labels={
    (u, v): f"{w} km"
    for u, v, w in G.edges(
        data="weight"
    )
},
font_color="red"
)

plt.title(
    "A* Search Route - Thane to CSMT"
)

plt.axis("off")

plt.show()

def rbfs(
    node,
    goal,
    f_limit,
    path,
    g
):
    if node == goal:
        return path, g, g

    successors = []

    for nxt, cost in graph[node].items():
        if nxt not in path:
            new_g = g + cost

            f = max(
                new_g + h[nxt],
                g + h[node]
            )

            successors.append(
                [nxt, new_g, f]
            )

    if not successors:
        return (
            None,
            float("inf"),
            float("inf")
        )

    while True:
        successors.sort(
            key=lambda x: x[2]
        )

```



```

    key=lambda x: x[2]
)

best = successors[0]

if best[2] > f_limit:

    return (
        None,
        float("inf"),
        best[2]
    )

alternative = (
    successors[1][2]
    if len(successors) > 1
    else float("inf")
)

result, distance, new_f = rbfs(
    best[0],
    goal,
    min(
        f_limit,
        alternative
    ),
    path + [best[0]],
    best[1]
)

best[2] = new_f

result,
distance,
new_f
)

path_r, distance_r, _ = rbfs(
    "Thane",
    "CSMT",
    float("inf"),
    ["Thane"],
    0
)

print("\nRecursive Best-First Search")

print("Optimal Path Found:")

print(" -> ".join(path_r))

print(
    "Total Distance =",
    distance_r,
    "km"
)

plt.figure(figsize=(12, 8))

nx.draw_networkx_nodes(
    G,
    pos,

```

SHETH L.U.J. AND SIR M.V. COLLEGE

```
T090_PRACTICAL2_Alpy - C:\practical\AI\T090_PRACTICAL2_Alpy (3.13.0)
File Edit Format Run Options Window Help

G,
pos,
node_color="lightblue",
node_size=2500
)

nx.draw_networkx_edges(
G,
pos,
edge_color="gray"
)

path_edges = list(
zip(path_r, path_r[1:])
)

nx.draw_networkx_edges(
G,
pos,
edgelist=path_edges,
edge_color="green",
width=4
)

nx.draw_networkx_labels(
G,
pos,
labels={
n: f"{n}\n(h={h[n]})"
for n in G.nodes()
},
font_size=9,

```

Activate Windows
Go to Settings to activate Windows.

Ln: 543 Col: 0

29°C
Partly cloudy

Search

ENG
IN

20:13
03-09-2026

```
T090_PRACTICAL2_Alpy - C:\practical\AI\T090_PRACTICAL2_Alpy (3.13.0)
File Edit Format Run Options Window Help

nx.draw_networkx_labels(
G,
pos,
labels={
n: f"{n}\n(h={h[n]})"
for n in G.nodes()
},
font_size=9,
font_weight="bold"
)

nx.draw_networkx_edge_labels(
G,
pos,
edge_labels={
(u, v): f"{w} km"
for u, v, w in G.edges(
data="weight"
)
},
font_color="red"
)

plt.title(
"RBFS Route - Thane to CSMT"
)

plt.axis("off")

plt.show()
```

Activate Windows
Go to Settings to activate Windows.

Ln: 543 Col: 0

29°C
Partly cloudy

Search

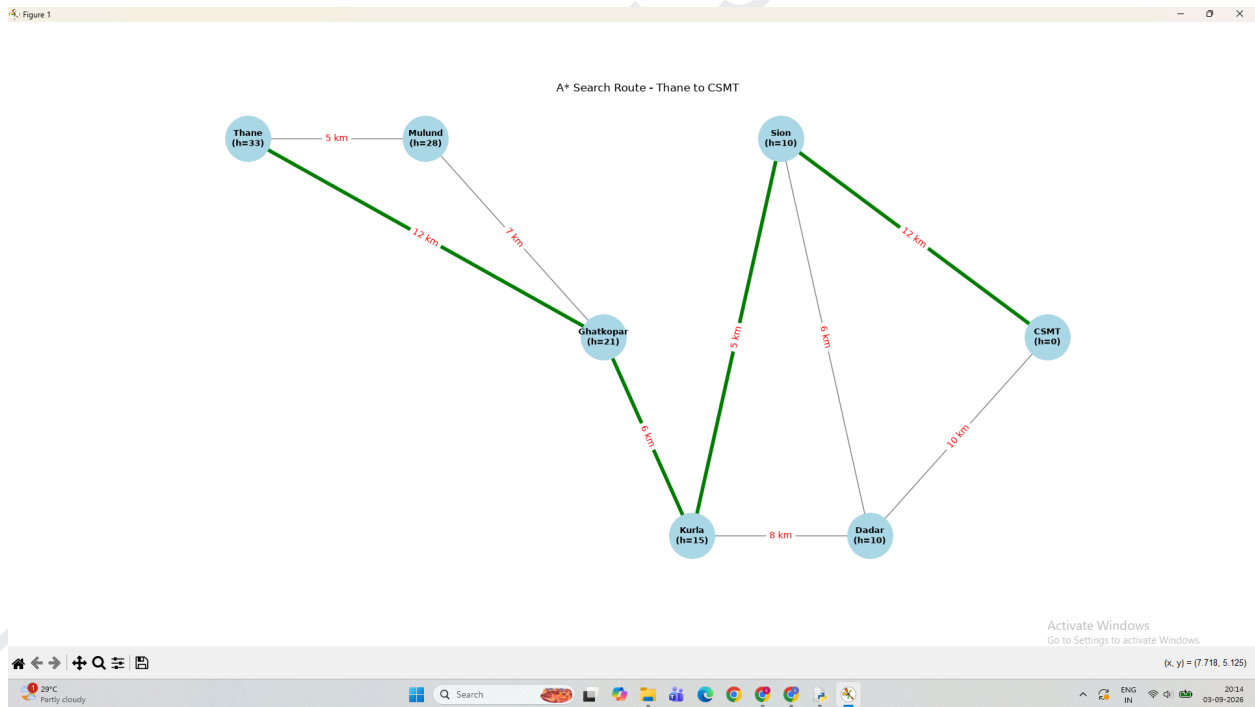
ENG
IN

20:13
03-09-2026

SHETH L.U.J. AND SIR M.V. COLLEGE

OUTPUT:

```
Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/practical/AI/T090_PRACTICAL2_AI.py =====
A* Search
Optimal Path Found:
Thane -> Ghatkopar -> Kurla -> Sion -> CSMT
Total Distance = 35 km
```



Compare the performance and effectiveness of both algorithms

Metric	A* Search	Recursive Best-First Search
Search Method	Priority Queue	Recursion
Evaluation Function	$f(n) = g(n) + h(n)$	Uses f-limit
Memory Usage	High	Low
Search Strategy	Best-first	Recursive best-first
Backtracking	Not required	Required
Completeness	Yes	Yes
Optimality	Yes, with admissible heuristic	Yes, with admissible heuristic
Route	Borivali → Goregaon → Bandra → Dadar → Churchgate	Borivali → Goregaon → Bandra → Dadar → Mumbai Central → Churchgate
Total Distance	37 km	37 km